INTRODUCTION TO PROGRAMMING PARADIGMS

Introduction to Programming Methodologies

- A programming methodology is just a programming practice to help you plan and structure your code in a more defined way.
- **Programming methodology is a process of developing programs that involves strategically dividing important tasks into functions to be utilized by the entirety of the program. It also describes the thinking process that goes into developing a programming solution for a specific problem.**
- When programs are developed to solve real-life problems like inventory management, payroll processing, student admissions, examination result processing, etc. they tend to be huge and complex.
- The approach to analyzing such complex problems, planning for software development and controlling the development process is called **programming methodology**.
- In other words, programming methodology deals with the analysis, design and implementation of programs.

Why Study Programming Methodologies?

- A programming methodology deals with providing a way to consider and manage the development, design, implementation, and testing of a piece of software.
- For example one common methodology is the Top down approach where you look at the big picture, what do you want to make in the end and break that down into more detailed sub parts until you have a complete understanding of the system. In contrast to that a Bottom up approach looks at combining small parts into a more complex and complete system.
- Studying Programming Methodology, you will be able to
  - *"think like a programmer."*
  - *approach problem solving from a computational perspective, and gain exposure to different areas in Computer Science.*
  - *have a firm grasp of how a programmer should "think," because you will spend a lot of time analyzing, designing, and implementing programs.*
- This is a fundamental skill that will help you tackle more difficult problems and programs in the future.

Types of Programming Methodologies

- Procedure-Oriented (Procedural) Programming
- Object-Oriented Programming
- Functional Programming
- Logical Programming
- Top-down or Modular Approach
- Bottom-up Approach

Procedural Programming

- The functionality of the computer program is divided in “procedures”
  These “procedures” are block of logic that perform a certain set of actions that are grouped together.
- In other words, problem is broken down into procedures, or blocks of code that perform one task each. All procedures taken together form the whole program. It is suitable only for small programs that have low level of complexity.
- **Example** – For a calculator program that does addition, subtraction, multiplication, division, square root and comparison, each of these operations can be developed as separate procedures. In the main program each procedure would be invoked on the basis of user’s choice.

Object-Oriented Programming

- The functionality of the computer program is represented by “objects”
  Each “object” has its own internal process (methods), that can be called from “outside” by calling those methods with the appropriate input parameters. Objects can have a hierarchy and can inherit characteristics.
- In other words, the solution revolves around entities or objects that are part of problem. The solution deals with how to store data related to the entities, how the entities behave and how they interact with each other to give a cohesive solution.
- **Example** – If we have to develop a payroll management system, we will have entities like employees, salary structure, leave rules, etc. around which the solution must be built.

Functional Programming

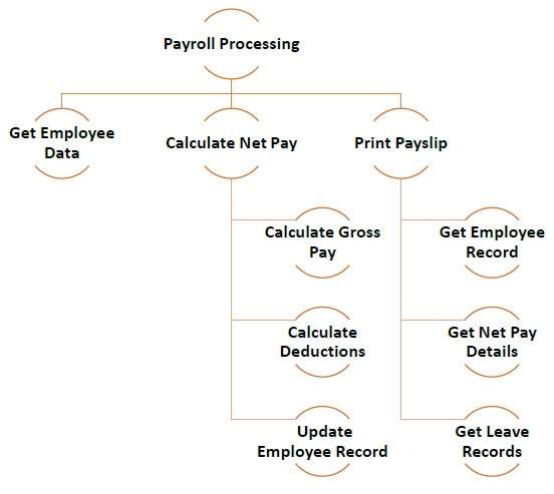
- Here the problem, or the desired solution, is broken down into functional units. Each unit performs its own task and is self-sufficient.
- These units are then stitched together to form the complete solution.
- **Example** – A payroll processing can have functional units like employee data maintenance, basic salary calculation, gross salary calculation, leave processing, loan repayment processing, etc.

Logical Programming

- Here the problem is broken down into logical units rather than functional units.
- **Example:** In a school management system, users have very defined roles like class teacher, subject teacher, lab assistant, coordinator, academic in-charge, etc. So the software can be divided into units depending on user roles. Each user can have different interface, permissions, etc.

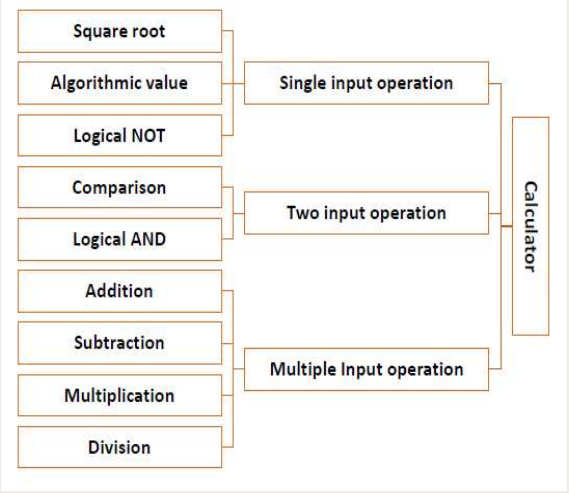
Top-down Approach

- The problem is broken down into smaller units, which may be further broken down into even smaller units.
- Each unit is called a **module**.
- Each module is a self-sufficient unit that has everything necessary to perform its task.
- The following illustration shows an example of how you can follow modular approach to create different modules while developing a payroll processing program.



Bottom-up Approach

- In bottom-up approach, system design starts with the lowest level of components, which are then interconnected to get higher level components.
- This process continues till a hierarchy of all system components is generated.
- However, in real-life scenario it is very difficult to know all lowest level components at the outset.
- So bottoms up approach is used only for very simple problems.
- Let us look at the components of a calculator program.



Modular Programming

- A real-life problem is complex and big. If a monolithic solution is developed it poses these problems –
  - *Difficult to write, test and implement one big program*
  - *Modifications after the final product is delivered is close to impossible*
  - *Maintenance of program very difficult*
  - *One error can bring the whole system to a halt*
- To overcome these problems, the solution should be divided into smaller parts called **modules**.
- The technique of breaking down one big solution into smaller modules for ease of development, implementation, modification and maintenance is called **modular technique** of programming or software development.

Advantages of Modular Programming

- Enables faster development as each module can be developed in parallel
- Modules can be re-used
- As each module is to be tested independently, testing is faster and more robust
- Debugging and maintenance of the whole program easier
- Modules are smaller and have lower level of complexity so they are easy to understand

Identifying the Modules

- Identifying modules in a software is a challenging task because there cannot be one correct way of doing so.
- Here are some pointers to identifying modules:
  - *If data is the most important element of the system, create modules that handle related data.*
  - *If service provided by the system is diverse, break down the system into functional modules.*
  - *If all else fails, break down the system into logical modules as per your understanding of the system during requirement gathering phase.*
- For coding, each module has to be again broken down into smaller modules for ease of programming.